



End-of-Turn Detection (120 minutes)

The problem

Every voice AI agent has to constantly decide at every pause from the human user: **is the human done talking, or just pausing in between?**

If the AI agent answers too early, the agent talks over humans. If it answers too late then the conversation fills with silence.

Your task is to build a model from scratch that predicts end of turn, from real audio data. Details below.

The data (eot_handout.zip)

You have a folder `eot_data/` with two subfolders:

```
None
eot_data/
  english/
    audio/          en__NNN.wav  (16 kHz mono, one user turn per
file)
    labels.csv
  hindi/
    audio/          hi__NNN.wav
    labels.csv
```

Each WAV is one real user turn from a human-to-voice-agent phone conversation (restaurant bookings, appointments, orders). Every silence pause of ≥ 100 ms inside the turn is annotated in `labels.csv`:

column	meaning
turn_id	which turn (matches the wav filename)
audio_file	relative path to the wav
pause_index	0, 1, 2, ... within the turn



pause_start	seconds - the moment speech stops
pause_end	seconds - the moment speech resumes (or file ends)
label	<code>hold</code> = user continues after this pause, <code>eot</code> = this pause is the true end of the turn

Your task

Write a model that, for **each pause**, outputs `p_eot`, the probability that the turn is over.

Causality rule (hard requirement): for a pause, your features may use ONLY audio from time 0 up to `pause_start` of that pause. Never audio after the pause. A live agent cannot hear the future. We will read your feature code to check this.

Deliverables (all five required)

1. `SUMMARY.html` - a detailed html using the coding agent, describing your solution, results, graphs and summary of the MD files below and what was done by human and what was done by coding agent and why it beats status quo.
2. `predict.py` - runs as: `python predict.py --data_dir <folder> --out predictions.csv` It must work on a folder with the same structure/labels schema it has never seen. `predictions.csv` columns: `turn_id`, `pause_index`, `p_eot`.
3. `predictions.csv` for BOTH provided language folders.
4. `RUNLOG.md` - after every scoring run: the score, and 1–2 lines on what you changed and why. This is graded.
5. `NOTES.md` - max 10 sentences: what signal your model uses, where it still fails, what you would do with one more day.

Scoring

Run `python score.py --data_dir eot_data/english --pred predictions.csv` (same for hindi). It simulates a live agent with your scores and reports:

Mean response delay (ms) at a false-cutoff rate $\leq 5\%$ - lower is better.

In plain words: tuned so the agent interrupts users at most 5% of the time, how long does it keep users waiting after they have actually finished?

Your final grade uses a **hidden test set** (unseen turns, mostly Hindi) on our machine, plus your run log and a 5-minute discussion of your model.



Reference points on this exact metric:

system	delay @ 5% cutoffs
silence-only baseline (given)	~1600 ms

Rules

- Laptop CPU only. No GPUs, no cloud training.
- Allowed libraries: numpy, scipy, scikit-learn, pandas, librosa, PyTorch.
- **NOT allowed:** any pretrained model or downloaded weights of any kind (no Whisper, wav2vec, Silero, webtrcvad, HuggingFace models, TTS/ASR APIs). No external datasets.
- AI coding assistants (Claude Code, Copilot, Codex, Cursor) **are allowed**. We calibrate for this: we know exactly what score they reach on their own.
 - Your grading is done based on what you add beyond what the coding agents do. If you just rely on them, you won't be able to get a good model working.

Suggested path (not mandatory)

- **0–10 min:** run `starter/baseline.py`, score it, LISTEN to a few wavs where `hold` pauses look long - understand why silence alone fails.
- **10–35 min:** extract prosodic features from the last ~1.5 s of speech before each pause (`starter/features.py` has audio loading and a pitch tracker). Train a small classifier. Score it.
- **35–55 min:** look at your worst errors. Fix what you see. Iterate.
- **55–60 min:** finalize predictions, create SUMMARY.html, RUNLOG.md, NOTES.md.

Good luck.